
DATA STRUCTURES ASSIGNMENT 01

1)Sum of two matrices

CODE:

```
#include<stdio.h>

int main(){
    int A[10][10],B[10][10],C[10][10];
    int n,i,j;
    printf("Enter the size of the matrix:");
    scanf("%d",&n);
    printf("Enter the elements of matrix 1:");
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            scanf("%d",&A[i][j]);
        }
    }
    printf("Enter the elements of matrix 2:");
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            scanf("%d",&B[i][j]);
        }
    }
    printf("The sum of two matrices:\n");
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            C[i][j]=A[i][j]+B[i][j];
        }
    }
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            printf("%d\t",C[i][j]);
```

```

        printf("\n");
    }
    return 0;
}

```

OUTPUT:

Output

Clear

```

Enter the size of the matrix:2
Enter the elements of matrix 1:5
4
3
2
Enter the elements of matrix 2:1
2
3
4
The sum of two matrices:
6  6
6  6

=== Code Execution Successful ===

```

2)Read n elements into an array and display them in reverse order

CODE:

```

#include<stdio.h>

int main(){
    int A[10];
    int n,i;
    printf("Enter number of elements:");
    scanf("%d",&n);
    printf("Enter elements of the array:");
    for(i=0;i<n;i++)
        scanf("%d",&A[i]);
    printf("Original array:\n");
    for(i=0;i<n;i++)

```

```

        printf("%d\t",A[i]);

printf("\nReversed array:\n");

for(i=n-1;i>=0;i--)

    printf("%d\t",A[i]);

return 0;

}

```

OUTPUT:

Output

Clear

```

Enter number of elements:7
Enter elements of the array:1
2
3
40
5
60
7
Original array:
1  2  3  40  5  60  7
Reversed array:
7  60  5  40  3  2  1

=== Code Execution Successful ===

```

3)Count the total number of duplicate elements in an array

CODE:

```

#include<stdio.h>

int main(){

    int A[10];

    int n,i,j,count=0;

    printf("Enter number of elements:");

    scanf("%d",&n);

    printf("Enter the elements of the array:");

    for(i=0;i<n;i++)

        scanf("%d",&A[i]);

```

```

for(i=0;i<n;i++){
    for(j=i+1;j<n;j++){
        if(A[i]==A[j]){
            count++;
            break;
        }
    }
}

printf("The total number of duplicate elements:%d",count);

return 0;
}

```

OUTPUT:

Output

Clear

```

Enter number of elements:5
Enter the elements of the array:12
15
46
64
46
The total number of duplicate elements:1

=== Code Execution Successful ===

```

4)Print unique elements in an array

CODE:

```

#include<stdio.h>

int main(){
    int A[10];
    int n,i,j,count;
    printf("Enter the number of elements:");
    scanf("%d",&n);
    printf("Enter the elements of the array:");
    for(i=0;i<n;i++)

```

```

        scanf("%d",&A[i]);
printf("Unique elements:\n");
for(i=0;i<n;i++){
    count=0;
    for(j=0;j<n;j++){
        if(A[i]==A[j])
            count++;
    }
    if(count==1)
        printf("%d\t",A[i]);
}
return 0;
}

```

OUTPUT:

Output

Clear

```

Enter the number of elements:5
Enter the elements of the array:1
2
2
5
4
Unique elements:
1  5  4

=== Code Execution Successful ===

```

5)Insert element into sorted array

CODE:

```
#include <stdio.h>
```

```
int main()
{
    int a[100], n, i, element;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter the sorted array elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    printf("Enter the element to insert: ");
    scanf("%d", &element);

    for(i = n - 1; i >= 0 && a[i] > element; i--)
    {
        a[i + 1] = a[i];
    }

    a[i + 1] = element;
    n++;

    printf("Array after insertion:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
}
```

```

    }

    return 0;
}

```

OUTPUT:

Output

Clear

```

Enter the number of elements: 4
Enter the sorted array elements:
22
52
64
89
Enter the element to insert: 24
Array after insertion:
22 24 52 64 89

=== Code Execution Successful ===

```

6)Insert an element into an unsorted array

CODE:

```

#include <stdio.h>

int main()
{
    int a[100], n, i, pos, element;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter the array elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
}

```

```

    }

    printf("Enter the position to insert (1 to %d): ", n + 1);
    scanf("%d", &pos);

    printf("Enter the element to insert: ");
    scanf("%d", &element);

    for(i = n - 1; i >= pos - 1; i--)
    {
        a[i + 1] = a[i];
    }

    a[pos - 1] = element;
    n++;

    printf("Array after insertion:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

    return 0;
}

```

OUTPUT:

Output

Clear

```
Enter the number of elements: 5
Enter the array elements:
14
25
7
1
28
Enter the position to insert (1 to 6): 3
Enter the element to insert: 52
Array after insertion:
14 25 52 7 1 28
```

=== Code Execution Successful ===

7)Matrix multiplication

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a[10][10], b[10][10], c[10][10];
```

```
    int r1, c1, r2, c2;
```

```
    int i, j, k;
```

```
    printf("Enter rows and columns of first matrix: ");
```

```
    scanf("%d%d", &r1, &c1);
```

```
    printf("Enter rows and columns of second matrix: ");
```

```
    scanf("%d%d", &r2, &c2);
```

```
    if(c1 != r2)
```

```
    {
```

```
        printf("Matrix multiplication is not possible.");
```

```
    return 0;
}
```

```
printf("Enter elements of first matrix:\n");
for(i = 0; i < r1; i++)
{
    for(j = 0; j < c1; j++)
    {
        scanf("%d", &a[i][j]);
    }
}
```

```
printf("Enter elements of second matrix:\n");
for(i = 0; i < r2; i++)
{
    for(j = 0; j < c2; j++)
    {
        scanf("%d", &b[i][j]);
    }
}
```

```
for(i = 0; i < r1; i++)
{
    for(j = 0; j < c2; j++)
    {
        c[i][j] = 0;

        for(k = 0; k < c1; k++)
        {
```

```
        c[i][j] = c[i][j] + a[i][k] * b[k][j];
    }
}
}
```

```
printf("Resultant Matrix:\n");
for(i = 0; i < r1; i++)
{
    for(j = 0; j < c2; j++)
    {
        printf("%d ", c[i][j]);
    }
    printf("\n");
}
```

```
    return 0;
}
```

OUTPUT:

Output

Clear

```
Enter rows and columns of first matrix: 2
2
Enter rows and columns of second matrix: 2
2
Enter elements of first matrix:
1
2
5
4
Enter elements of second matrix:
3
2
1
2
Resultant Matrix:
5 6
19 18
```

=== Code Execution Successful ===

8)Matrix transpose

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a[10][10];
```

```
    int r, c, i, j;
```

```
    printf("Enter the number of rows and columns: ");
```

```
    scanf("%d%d", &r, &c);
```

```
    printf("Enter the matrix elements:\n");
```

```
    for(i = 0; i < r; i++)
```

```
    {
```

```

        for(j = 0; j < c; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }

    printf("Original Matrix:\n");
    for(i = 0; i < r; i++)
    {
        for(j = 0; j < c; j++)
        {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }

    printf("Transpose of the Matrix:\n");
    for(i = 0; i < c; i++)
    {
        for(j = 0; j < r; j++)
        {
            printf("%d ", a[j][i]);
        }
        printf("\n");
    }

    return 0;
}

```

OUTPUT:

Output

Clear

Enter the number of rows and columns: 3

2

Enter the matrix elements:

5

2

15

32

69

96

Original Matrix:

5 2

15 32

69 96

Transpose of the Matrix:

5 15 69

2 32 96

=== Code Execution Successful ===

9) Find the Sum of the Right (Main) Diagonal of a Matrix

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a[10][10], n;
```

```
    int i, j, sum = 0;
```

```
    printf("Enter the order of the matrix: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the matrix elements:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        for(j = 0; j < n; j++)
```

```
        {
```

```
            scanf("%d", &a[i][j]);
```

```

    }
}

for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
    {
        if(i == j)
        {
            sum = sum + a[i][j];
        }
    }
}

printf("Sum of the main diagonal = %d", sum);

return 0;
}

```

OUTPUT:

Output

Clear

```

Enter the order of the matrix: 3
Enter the matrix elements:
1
0
0
0
1
0
0
0
1
Sum of the main diagonal = 3

=== Code Execution Successful ===

```

10) Check whether the given matrix is an identity matrix

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a[10][10], n, i, j;
```

```
    int flag = 1;
```

```
    printf("Enter the order of the matrix: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the matrix elements:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        for(j = 0; j < n; j++)
```

```
        {
```

```
            scanf("%d", &a[i][j]);
```

```
        }
```

```
    }
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        for(j = 0; j < n; j++)
```

```
        {
```

```
            if(i == j)
```

```
            {
```

```
                if(a[i][j] != 1)
```

```
                {
```



```

        flag = 0;
    }
}
else
{
    if(a[i][j] != 0)
    {
        flag = 0;
    }
}
}
}

if(flag == 1)
    printf("The matrix is an identity matrix.");
else
    printf("The matrix is not an identity matrix.");

return 0;
}

```

OUTPUT:

Output

Clear

```

Enter the order of the matrix: 2
Enter the matrix elements:
1
0
0
1
The matrix is an identity matrix.

=== Code Execution Successful ===

```

11)Count the number of vowels, consonants , whitespaces and digits in a string

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char str[100];
```

```
    int i;
```

```
    int vowels = 0, consonants = 0;
```

```
    int digits = 0, spaces = 0;
```

```
    printf("Enter a string: ");
```

```
    fgets(str, sizeof(str), stdin);
```

```
    for(i = 0; str[i] != '\0'; i++)
```

```
    {
```

```
        if(str[i] == 'a' || str[i] == 'e' ||
```

```
           str[i] == 'i' || str[i] == 'o' ||
```

```
           str[i] == 'u' ||
```

```
           str[i] == 'A' || str[i] == 'E' ||
```

```
           str[i] == 'I' || str[i] == 'O' ||
```

```
           str[i] == 'U')
```

```
        {
```

```
            vowels++;
```

```
        }
```

```
        else if((str[i] >= 'a' && str[i] <= 'z') ||
```

```
                (str[i] >= 'A' && str[i] <= 'Z'))
```

```
        {
```

```

        consonants++;
    }
    else if(str[i] >= '0' && str[i] <= '9')
    {
        digits++;
    }
    else if(str[i] == ' ')
    {
        spaces++;
    }
}

printf("Vowels = %d\n", vowels);
printf("Consonants = %d\n", consonants);
printf("Digits = %d\n", digits);
printf("White spaces = %d\n", spaces);

return 0;
}

```

OUTPUT:

Output	Clear
<pre> Enter a string: rekha is a good girl Vowels = 7 Consonants = 9 Digits = 0 White spaces = 4 === Code Execution Successful === </pre>	

12)Frequency of characters in a string

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char str[100];
```

```
    int i, j, count;
```

```
    printf("Enter a string: ");
```

```
    fgets(str, sizeof(str), stdin);
```

```
    for(i = 0; str[i] != '\0'; i++)
```

```
    {
```

```
        count = 1;
```

```
        if(str[i] == ' ' || str[i] == '\n')
```

```
            continue;
```

```
        for(j = i + 1; str[j] != '\0'; j++)
```

```
        {
```

```
            if(str[i] == str[j])
```

```
            {
```

```
                count++;
```

```
            }
```

```
        }
```

```
        for(j = 0; j < i; j++)
```

```
        {
```

```
            if(str[i] == str[j])
```

```

        {
            count = 0;
            break;
        }
    }

    if(count != 0)
    {
        printf("%c = %d\n", str[i], count);
    }
}

return 0;
}

```

OUTPUT:

Output

Clear

```

Enter a string: banana
b = 1
a = 3
n = 2

=== Code Execution Successful ===

```

13)Count the number of words in a string

CODE:

```
#include <stdio.h>
```

```
int main()
```

```

{
    char str[100];

```

```
int i, words = 0;
```

```
printf("Enter a string: ");
```

```
fgets(str, sizeof(str), stdin);
```

```
for(i = 0; str[i] != '\0'; i++)
```

```
{
```

```
    if(str[i] != ' ' && str[i] != '\n')
```

```
    {
```

```
        if(i == 0 || str[i - 1] == ' ')
```

```
        {
```

```
            words++;
```

```
        }
```

```
    }
```

```
}
```

```
printf("Number of words = %d", words);
```

```
return 0;
```

```
}
```

OUTPUT:

Output

Clear

Enter a string: hello world this is rekha
Number of words = 5

=== Code Execution Successful ===

14) Sort the array of strings

CODE:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char str[10][50], temp[50];
```

```
    int n, i, j;
```

```
    printf("Enter the number of strings: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the strings:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        scanf("%s", str[i]);
```

```
    }
```

```
for(i = 0; i < n - 1; i++)
{
    for(j = 0; j < n - i - 1; j++)
    {
        if(strcmp(str[j], str[j + 1]) > 0)
        {
            strcpy(temp, str[j]);
            strcpy(str[j], str[j + 1]);
            strcpy(str[j + 1], temp);
        }
    }
}
```

```
printf("Strings in sorted order:\n");
```

```
for(i = 0; i < n; i++)
{
    printf("%s\n", str[i]);
}
```

```
return 0;
```

```
}
```

OUTPUT:

Output

Clear

Enter the number of strings: 3

Enter the strings:

rekha

jane

ram

Strings in sorted order:

jane

ram

rekha

=== Code Execution Successful ===

15)Toggle case of a sentence (lowercase to upper and vice versa)

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char str[100];
```

```
    int i;
```

```
    printf("Enter a sentence: ");
```

```
    fgets(str, sizeof(str), stdin);
```

```
    for(i = 0; str[i] != '\0'; i++)
```

```
    {
```

```
        if(str[i] >= 'a' && str[i] <= 'z')
```

```
        {
```

```
            str[i] = str[i] - 32;
```

```
        }
```

```
        else if(str[i] >= 'A' && str[i] <= 'Z')
```

```
        {
```

```
        str[i] = str[i] + 32;
    }
}

printf("Toggled sentence:\n");
printf("%s", str);

return 0;
}
```

OUTPUT:

Output

Clear

Enter a sentence: rekha GOOD
Toggled sentence:
REKHA good

=== Code Execution Successful ===